L-Expr: Vorrang und Assoziativität

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Java-artige Infix-Syntax: Vorrangregeln

- `(* (+ 1 2) 3)` ist immer eindeutig
- `1+2*3 == 1+(2*3) != (1+2)*3`
- Multiplikation/Division binden **stärker** als Addition/Subtraktion.

Tafel: Unterschiede im AST

Assoziativität (Klammerung)

`+`, `-`, `*`, `/` sind **linksassoziativ**:

```
1 - 2 - 3   =   (1 - 2) - 3
8 / 4 / 2   =   (8 / 4) / 2
1 + 2 + 3   =   (1 + 2) + 3
```

Vorrang und Assoziativität: Gestufte Grammatik-Regeln

```
expr  ::= term ('+' term)*
term  ::= atom ('*' atom)*
atom  ::= '-' atom
        | INT
```

Beispiel Parsen “1+2*3” und “1+2+3”

Wrap-up

- In Infix-Notation brauchen wir:
  - **Vorrang**: `*` und `/` binden stärker als `+` und `-`
  - **Assoziativität**: `+`, `-`, `*`, `/` sind linksassoziativ, z.B. `1-2-3 = (1-2)-3`
- **Gestufte konkrete Grammatik** (`expr -> term -> atom`) modelliert Vorrang:
  - `expr`: Ebene mit `+` und `-`
  - `term`: Ebene mit `*` und `/`
  - `atom`: Ebene mit Literalen, Klammern und unärem Minus
- Muster `X ::= Y (op Y)*` erzeugt **linksassoziative** Bäume



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Last modified: *a9e7254 2026-11-14 lexpr: add new screencast (precedence)*